

Learning Lean 4

Joshua W. Kelly

Friday 17th July, 2026



1 Basic Syntax

1.1 Basic syntax

The runnable version of this chapter lives in [LearningLean/BasicSyntax.lean](#).

1.1.1 Comments

```
- - This is a single -line comment.

/ -
This is a multi -line comment.
Here is another line.
- /
```

1.1.2 Inspecting terms and types with #check

#check asks Lean for the type of an expression without evaluating it.

```
#check true          - - Bool
#check 42             - - Nat
#check "Hello, World!" - - String
#check Type           - - Type 1
#check Nat            - - Type
#check ['h', 'e', 'l', 'l', 'o'] - - List Char
```

1.1.3 Printing definitions with #print

#print shows how something is defined.

```
#print Bool
```

Type itself cannot be printed with #print, because it is a type rather than a definition.

1.1.4 Definitions and functions

```
def pi : Float := 3.1415926
```

```
#check pi
```

```
- - An anonymous function (lambda) taking two natural numbers.
#check \lambda (a b : Nat) => a + b
```

2 Natural Deduction

2.1 Natural deduction

The Lean proofs for this chapter live in [LearningLean/NaturalDeduction.lean](#), inside the `Exercises` namespace.

2.1.1 5.5 Reduction to the absurd. $P \Rightarrow Q, QP$

This is a very useful technique. Validity of $P \Rightarrow Q, QP$ is proved with:

1	$P \Rightarrow Q$	
2	$\neg Q$	
3	$\begin{array}{ l} P \end{array}$	H
4	$\begin{array}{ l} Q \end{array}$	$E\Rightarrow 1,3$
5	$\begin{array}{ l} \neg Q \end{array}$	IT 2
6	$\neg P$	$I\neg 3,4,5$

The sub-derivation (lines 3–5) discharges the hypothesis P : assuming P , [$\Rightarrow$ -elimination] on lines 1 and 3 gives Q , while line 2 reiterates Q . The contradiction lets us introduce P ($\neg$ -introduction over lines 3, 4, 5).

In Lean this is theorem T55, where $\neg P$ is definitionally $P \rightarrow False$, so the whole proof is just function composition.

```
theorem T55 (h1 : P  $\rightarrow$  Q) (h2 :  $\neg$ Q) :  $\neg$ P :=  
  (h2 h1)
```

2.1.2 Conjunction introduction. $P, P \Rightarrow Q \vdash P \wedge Q$

Theorem T51 builds the pair proof of P , proof of Q directly, using the anonymous-constructor notation `_, _`:

```
theorem T51 (h1 : P) (h2 : P  $\rightarrow$  Q) : P  $\wedge$  Q :=  
  h1, h2 h1
```